

Progress Report — Weeks 2–7

Real-Time Anomaly Detection Platform for Branch Teller Operations
Retrait · Versement · Virement · Remise de Chèques

Yassine — Amen Bank, Direction Centrale du Système d'Information

Département des Projets Agence

Contents

- 1 Executive Summary
- 2 Source Extraction Completion
 - 2.1 CTAF 2021 Annual Report — Case Studies
 - 2.2 Structural Extractions
 - 2.3 Architectural Decisions D19–D22
 - 2.4 Source Phase Closure
- 3 Archetype Design and Calibration
 - 3.1 Five Client Archetypes
 - 3.2 Law n°41-2024 Impact
- 4 Synthetic Data Generator
 - 4.1 Architecture
 - 4.2 Validation Results
- 5 Data Infrastructure
 - 5.1 Generator Extensions
 - 5.2 PostgreSQL Cold Store
 - 5.3 Batch Profile Computation
- 6 Training Data Pipeline
 - 6.1 Point-in-Time Profile Design
 - 6.2 Employee Profile Design
 - 6.3 Batch Enrichment
 - 6.4 Jensen-Shannon Divergence for Behavioral Drift
- 7 Model Training — Initial Attempt and Root Cause Analysis
 - 7.1 Initial Results
 - 7.2 Root Cause Diagnosis
 - 7.3 Pipeline Rebuild
- 8 Score Fusion Layer
 - 8.1 Percentile Normalization
 - 8.2 Maturity-Adaptive Weighting
- 9 Decision Service
 - 9.1 Three-Layer Architecture
 - 9.2 Regulatory Rules
 - 9.3 Results
- 10 Batch Pipeline Realignment
- 11 SHAP Explainability Module
 - 11.1 Autoencoder Explainer (AEScorer)
 - 11.2 LSTM Explainer (LSTMScorer)
 - 11.3 Natural Language Generation
 - 11.4 Trigger-Aware Explanation Assembly
- 12 Streaming Pipeline — Phase 1: Infrastructure
 - 12.1 Docker Compose
 - 12.2 Centralized Configuration

- 12.3** Event Simulator
 - 12.4** Profile Hydration
 - 12.5** Enrichment Service
- 13** Streaming Pipeline — Phase 2: Scoring and Decision
 - 13.1** Integration Issues and Fixes
 - 13.2** Scoring Service
 - 13.3** Decision Service (Streaming Wrapper)
 - 13.4** Cold Start Resolution
 - 13.5** End-to-End Validation
- 14** Data Flow Summary
- 15** Key Technical Learnings
- 16** Artifacts Produced
- 17** Next Steps

1 Executive Summary

This report covers three weeks of development during which the project progressed from source extraction and data pipeline design through model training and evaluation to a fully operational real-time streaming pipeline. The work can be divided into three phases:

- **Weeks 2–3 — Data foundation:** Completed regulatory source extraction (CTAF), designed five client archetypes, built a synthetic data generator producing 340,839 labeled transactions, stood up the PostgreSQL cold store, and implemented batch profile computation producing 260,000 weekly client profiles and 7,800 employee profiles.
- **Weeks 4–5 — ML pipeline:** Trained Autoencoder and LSTM models, diagnosed and resolved a fundamental feature engineering flaw (profile dominance over per-event signal), rebuilt the enrichment pipeline around 30 contrast features, designed the Score Fusion layer with maturity-adaptive weighting, and implemented a three-layer Decision Service with eight regulatory rules.
- **Weeks 6–7 — Production pipeline:** Implemented SHAP explainability for both models, containerized the streaming infrastructure (RedPanda + Redis), built three streaming services (Enrichment, Scoring, Decision), resolved cold-start and schema integration issues, and validated the complete pipeline end-to-end.

Key metrics: Autoencoder test AUC 0.9853, LSTM test AUC 0.9866, fused AUC 0.9725, Decision Service detection rate 93.7% at REVIEW tier, BLOCK precision 71%, and a validated streaming pipeline producing a healthy tier distribution of approximately 70% INFO / 15% REVIEW / 15% BLOCK.

2 Source Extraction Completion

2.1 CTAF 2021 Annual Report — Case Studies

Four typological case studies were extracted from the CTAF (Commission Tunisienne des Analyses Financières) 2021 Annual Report using a structured 7-field template. Each case surfaced schema amendments across three layers (raw event, client profile, enriched event):

- **Case 1 (Espèces-heavy structuring):** Multiple versements just below the 10,000 DT reporting threshold, dispersed across branches. Informed the near-threshold ratio feature and branch distribution drift detection.
- **Case 2 (Virement chain):** Series of virements to unknown beneficiaries with round amounts. Informed first-time beneficiary detection and round-amount flagging.
- **Case 3 (PPE exploitation):** Politically exposed person used as a conduit. Validated the PPE binary flag layered on archetypes.
- **Case 4 (Remise de chèques laundering):** Third-party cheques deposited in rapid succession. Informed cumulative amount tracking and cheque-specific anomaly patterns.

2.2 Structural Extractions

Six quantitative extraction targets were completed:

- Instrument co-occurrence rates: espèces 68%, virement 56%, chèque 34%
- BBE concentration: 85% of operations in the 5,000–20,000 DT band

- Predicate offence taxonomy: contrebande 20%, corruption 19%, escroquerie 11%
- Geographic concentration across Tunisian governorates
- Two directly encodable regulatory rules

2.3 Architectural Decisions D19–D22

Decision	Description
D19	Evidence-gated sector risk scoring — sector risk weights backed by CTAF data, non-evidenced sectors scored at zero
D20	regulatory_client_type (occasional/habitual) and maturity_status (new/mature) as independent fields
D21	Multi-tier anomaly difficulty gradient (easy/medium/hard) for controlled injection
D22	Multi-operation anomaly sequences as the injection format for hard scenarios

2.4 Source Phase Closure

BCT (Banque Centrale de Tunisie) circulars 2018-09, 2017-08, and 2021-05 were confirmed as restricted documents not publicly available. The source extraction phase was closed with three sources: S1 (PaySim), S2 (Amen Bank 2023 Annual Report), and S3 (CTAF 2021). BCT circulars are documented as a known gap.

3 Archetype Design and Calibration

3.1 Five Client Archetypes

Five archetypes were designed across six behavioral dimensions (operation mix, frequency, timing, amount ranges, counterparty patterns, branch loyalty), calibrated using Amen Bank’s deposit composition data:

Archetype	Population	Dominant Operation	Monthly Volume	Amount Range (DT)
Salaried	49%	Retrait (50%)	~5.7 ops	200–500
Small Business	33%	Versement (55%)	~37.5 ops	500–2,000
Student	5%	Retrait (95%)	~7.2 ops	40–80
Retiree	7%	Retrait/Virement	~3.2 ops	200–500
Big Business	6%	Virement (78%)	~63 ops	2,000–20,000

Population proportions derived from Amen Bank’s 61% individual / 33% private business / 6% institutional split, with the individual bucket subdivided ~80% salaried / 12% retiree / 8% student.

3.2 Law n°41-2024 Impact

A regulatory discovery during calibration: Law n°41-2024 (effective February 2025) caps individual cheques at 30,000 DT, confirmed by BCT Q1 2026 data showing a 24.9% volume collapse in cheque usage. This revised the big business archetype: remise de chèques dropped from

~12–15/month to ~2–4/month, with virement absorbing the difference. A new anomaly scenario was added: cheque structuring just under the 30,000 DT ceiling.

4 Synthetic Data Generator

4.1 Architecture

The generator follows a 4-class, 2-phase design: Archetype (Layer 1) loads behavioral parameters from YAML; Client (Layer 2) samples a fixed personality via Gaussian sampling with clipping; Generator (orchestrator) plans anomaly windows then generates events; Event generation adds per-event noise and payload construction. The two-phase approach (planning then generation) ensures anomaly injection is controlled and reproducible.

4.2 Validation Results

At 10,000 clients over 180 simulated days:

Metric	Value
Total events	243,904 (after generator rebuild)
Anomaly rate	0.147% (target: 0.13%, acceptable)
Anomaly scenarios	9 well-implemented scenarios across 3 difficulty tiers
Clients	10,000 across 5 archetypes
Employees	300 (2 per branch, branch-linked)
Branches	150

5 Data Infrastructure

5.1 Generator Extensions

The generator was extended to produce four CSV files per run: transactions.csv (243,904 rows), clients_master.csv (10,000), accounts.csv (10,000), and employees_master.csv (300). Key additions include branch-linked employee assignment (2 per branch, validated with 0 cross-branch violations), account_opening_date per client sampled from archetype-calibrated ranges, and client_type regulatory classification (personne_physique vs personne_morale).

5.2 PostgreSQL Cold Store

PostgreSQL 17 with database amen_anomaly. Tables created with foreign-key-respecting load order: clients_master → accounts → employees_master → transactions → profile_snapshots. Data loaded via psycopg2's copy_expert using PostgreSQL's COPY protocol. Indexes on (client_id, timestamp) and (employee_id, timestamp) for efficient profile computation.

5.3 Batch Profile Computation

The batch profile script computes 122-field client profiles stored as JSONB in profile_snapshots:

- **Category 1 — Personal info (5 fields):** archetype, client_type, home_branch_id, account_age_days, maturity_status
- **Category 2 — Transaction stats (96 fields):** 4 operations × 4 windows (24h/7d/30d/90d) × 6 statistics
- **Category 3 — Behavioral patterns (14 fields):** 5 distributions at 30d/180d plus known counterparty sets
- **Category 4 — Financial state (6 fields):** inflow/outflow/net at 30d/90d
- **Category 6 — LSTM buffer (1 field):** Last 50 events ordered chronologically

260,000 client profile snapshots (26 weeks × 10,000 clients), 7,800 employee profile snapshots (26 weeks × 300 employees), and 5 archetype baselines were computed.

6 Training Data Pipeline

6.1 Point-in-Time Profile Design

Training data must use point-in-time profiles to avoid data leakage. Using the final simulation profile (June 29) to enrich a January transaction would leak future information. The solution: weekly profile snapshots matched to transactions via `pd.merge_asof` — an as-of join selecting the most recent profile before each transaction timestamp.

6.2 Employee Profile Design

Employee profiles follow the same batch pattern with a key addition: peer comparison. With only 2 tellers per branch, per-branch z-scores are statistically meaningless. A split normalization approach was designed: volume metrics are normalized per-branch first then z-scored bank-wide (prevents high-volume branches from appearing as outliers), while ratio metrics (near-threshold ratio, error rate, client concentration) are z-scored directly bank-wide since ratios are already volume-independent.

6.3 Batch Enrichment

The batch enrichment script produces the complete training dataset by joining each transaction with its point-in-time profile and computing all derived features. Output: 340,839 enriched events × 74 columns, spanning 11 sections of the enriched event schema.

6.4 Jensen-Shannon Divergence for Behavioral Drift

JS-divergence measures how much a client's recent behavior (30-day window) has shifted from their long-term pattern (180-day window). Unlike comparing individual statistics, JS-divergence captures overall distribution shape change in a single scalar. Five drift features are computed: branch, hour, day-of-week, operation mix, and employee distribution.

7 Model Training — Initial Attempt and Root Cause Analysis

7.1 Initial Results

The Autoencoder was trained on normal-only enriched events using the 74-feature schema. Three configurations were evaluated using Optuna with AUC-ROC as the objective. Best AUC: 0.577. The LSTM used a multi-task prediction architecture with 8 heads. Best AUC: 0.564. Both models performed near random, triggering a root cause investigation.

7.2 Root Cause Diagnosis

Two independent root causes were identified:

- **Undetectable scenarios:** Many anomaly scenarios had stub implementations or produced no measurable change in the enriched feature space. For instance, `timing_anomaly` shifted the event hour, but the enriched event contained no per-event hour feature — only a profile-level hour distribution that absorbs single-event changes.
- **Profile dominance:** The 74-feature enriched event was dominated by raw profile field dumps (weekly snapshot means, stds, distributions) that were identical across all events from the same client in the same week. Per-event signal was drowned out by profile noise. This was identified during analysis: the profile was dominating the raw event, making enriched events nearly indistinguishable between normal and anomalous.

7.3 Pipeline Rebuild

Three components were redesigned:

- **Generator:** Reduced from 13 to 9 well-implemented scenarios with verified feature-space footprints. Three difficulty tiers: easy (`amount_spike`, `round_amount`, `duplicate_virement`), medium (`frequency_burst`, `cumulative_threshold`, `timing_anomaly`), hard (`smurfing`, `cheque_structuring`, `repeated_client_employee`).
- **Enrichment:** Complete redesign from profile dumps to contrast features. Each feature measures the deviation of the current event against the client's baseline. Feature count dropped from 74 to 30: 7 raw event features, 17 contrast features, 6 context features.
- **Training data:** Oracle profile shortcut — client personality parameters exported directly from the generator for exact baselines. To be replaced by batch-computed profiles in production.

Model	Old AUC	Rebuilt AUC
Autoencoder	0.577	0.9853
LSTM	0.564	0.9866

The improvement is entirely structural — contrast features expose per-event deviations that reconstruction-based models can detect.

8 Score Fusion Layer

8.1 Percentile Normalization

The AE and LSTM produce anomaly scores on fundamentally different scales (MSE reconstruction error vs prediction error). Each model's raw score is mapped to a percentile rank against a saved reference distribution computed from normal training data, placing both on a common [0, 1] scale where 0.95 means “higher than 95% of normal events.”

8.2 Maturity-Adaptive Weighting

A sigmoid function of `days_since_account_opening` (midpoint 90 days, steepness 15) controls the weight between models. New accounts favor the Autoencoder (works from the first event), mature accounts favor the LSTM (requires sequence history). `days_since_account_opening` was chosen over event count because event count penalizes low-frequency archetypes like retirees.

Fusion formula: $S_{\text{fused}} = w_{\text{lstm}} \cdot S_{\text{lstm}} + (1 - w_{\text{lstm}}) \cdot S_{\text{ae}}$. Fused AUC on validation set: 0.9725.

9 Decision Service

9.1 Three-Layer Architecture

The Decision Service is a stateless engine applying three layers in order:

- **Layer 1 — Score thresholds:** REVIEW (T1=0.95), ALERT (T2=0.99), BLOCK (T3=0.999), calibrated on the validation set
- **Layer 2 — Regulatory rules:** Eight rules that can escalate tiers (never de-escalate). All regulatory flags logged for audit regardless of escalation
- **Layer 3 — Risk history:** Recidivist escalation (prior alerts boost tier), false-positive dampening, cold-start conservatism

9.2 Regulatory Rules

Rule	Description	Escalation
1	BCT 10,000 DT reporting threshold (with <code>z_amount</code> ≥ 2 guard)	→ ALERT
2	Smurfing band (8,000–9,999 DT) with ≥ 2 near-threshold txns in 7d	→ ALERT
3	Cheque ceiling proximity (25,000–30,000 DT, Law n°41-2024)	→ REVIEW
4	Passeur de fonds: occasional client, ≥ 5 versements in 24h (CTAF)	→ ALERT
5	Large virement (≥ 5,000 DT) to new beneficiary	→ REVIEW
6	Duplicate transaction detection	→ REVIEW
7	Round amount pattern (≥ 5,000 DT) with high 24h cumulative	→ REVIEW
8	Frequency burst (≥ 5× expected daily rate)	→ REVIEW

9.3 Results

Metric	Value
Detection rate at REVIEW tier	93.7%
BLOCK precision	71%

Metric	Value
Alerts per branch per day	~1.7 at REVIEW threshold (~0.5 at ALERT)
Fused AUC	0.9725

10 Batch Pipeline Realignment

After the generator rebuild (new scenarios, branch-linked employees), all stored PostgreSQL data was from the old run with different client IDs and mismatched schema. The full batch pipeline was rebuilt: PostgreSQL schema dropped and recreated, data regenerated (243,904 events), models retrained, thresholds recalibrated, and batch profiles recomputed. A critical generator bug was fixed: employees were assigned from a global pool rather than being branch-linked, allowing a single employee to appear at multiple branches — a physical impossibility in branch banking. Post-fix LSTM AUC improved from 0.9430 to 0.9866 because branch-linked employees produce cleaner sequential patterns.

11 SHAP Explainability Module

The explainability module fulfills a core requirement from the sujet de stage: every alert must include a human-readable justification. The implementation provides a unified explanation framework across both ML models and the regulatory rule engine.

11.1 Autoencoder Explainer (AEScorer)

The AEScorer wraps the trained Autoencoder with SHAP's KernelExplainer. DeepExplainer was ruled out because the anomaly score involves a subtraction (reconstruction – input) outside the model's forward method, breaking forward-pass tracing. KernelExplainer treats the scoring function as a black box. A background dataset of 100 scaled normal events (models/shap_background.npy) is regenerated with each weekly model retrain. For each flagged event, the top-3 features by absolute SHAP value are reported with both SHAP contribution and raw value.

11.2 LSTM Explainer (LSTMScorer)

Raw SHAP over the LSTM's full input shape ($10 \times 30 = 300$ values) produces unmanageable noise. Instead, the LSTMScorer uses an expected-vs-actual comparison: the LSTM predicts what the next event should look like based on the client's sequence history, then reports the top-3 features with the largest deviation. A whitelist of 11 continuous, interpretable features filters out binary flags and z-scores. Example: "amount: expected 450.00 DT, actual 8,450.00 DT."

11.3 Natural Language Generation

An LLM-based approach was rejected due to bank security constraints, non-deterministic outputs, and unnecessary complexity. A deterministic template dictionary maps feature names to parameterized sentences, producing consistent and auditable explanations. Example: "This versement is suspect because the amount (8,450 DT) is 4.2× the client's average, and it is the 3rd deposit in 48 hours."

11.4 Trigger-Aware Explanation Assembly

The explanation assembler produces different structures depending on the alert trigger:

- **ML-only:** Fused score crossed threshold → AE SHAP + LSTM expected-vs-actual
- **Rule-only:** Regulatory rule escalated tier → rule description with specific values
- **Both:** Score set initial tier, rule escalated further → combined explanation

Validated end-to-end on a cheque structuring anomaly: BLOCK-tier alert with convergent explanations from all three mechanisms (AE flagged amount deviation, LSTM flagged sequence break, regulatory engine flagged cheque ceiling proximity).

12 Streaming Pipeline — Phase 1: Infrastructure

12.1 Docker Compose

Service	Image	Port(s)	Purpose
RedPanda	redpandadata/redpanda:latest	9092, 8082, 9644	Kafka-compatible broker (no JVM/ZooKeeper). Dual listener for container + host access.
Redis 7	redis:7-alpine	6379	Hot profile store and event buffer persistence.

12.2 Centralized Configuration

All configuration consolidated into `.env` + `src/config.py` using `python-dotenv`. Exposes `DB_CONFIG`, `REDIS_HOST/PORT`, and `KAFKA_BOOTSTRAP` from environment variables with sensible defaults, replacing hardcoded connection strings across multiple files.

12.3 Event Simulator

Reads transactions from PostgreSQL and publishes to `raw-events` with `client_id` as partition key (preserving LSTM temporal ordering). Supports replay mode (configurable speed factor) and burst mode (no delay, for load testing).

12.4 Profile Hydration

Required startup step loading hot data into Redis before streaming services start:

Redis Key Pattern	Count	Source
<code>profile:client:{id}</code>	10,000	<code>profile_snapshots</code> (latest per client)
<code>profile:employee:{id}</code>	300	<code>employee_profile_snapshots</code> (latest)
<code>baseline:{archetype}</code>	5	<code>archetype_baselines</code>
<code>buffer:client:{id}</code>	10,000	<code>recent_events_buffer</code> from profile (cold start fix)

12.5 Enrichment Service

Consumes from `raw-events`, computes 30 contrast features using Redis profiles and per-client/employee rolling buffers, produces to `enriched-events`. Rolling buffers persist in Redis (not service memory) for restart resilience. Client buffers retain 50 events; employee buffers 100. Validated with 500 events end-to-end.

13 Streaming Pipeline — Phase 2: Scoring and Decision

13.1 Integration Issues and Fixes

Integration testing revealed six profile key mismatches between the batch-computed profile schema and the streaming Enrichment Service's expected key names. All mismatches were caused by assumed key names that did not match the actual profile JSON structure:

Enrichment Lookup	Actual Profile Key	Fix
<code>amount_mean_{op}</code>	<code>tx_{op}_30d_mean</code>	Key name correction
<code>amount_std_{op}</code>	<code>tx_{op}_30d_std</code>	Key name correction

Enrichment Lookup	Actual Profile Key	Fix
op_mix_{op}	dist_30d_operation (nested dict)	Restructured lookup
home_branch	home_branch_id	Key name correction
account_opening_date	account_age_days (pre-computed int)	Use integer directly
preferred_day	Not in profile	Neutralized with training mean

Additionally, a **fallback chain** was implemented for amount statistics: when a client has no transactions of a given operation type in the 30-day window (common for infrequent operations), the z-score calculation falls back to the 90-day window, then to the archetype baseline. Without this, infrequent operations produced z-scores of 2,000+, saturating the anomaly score.

The LSTM sequence buffer (sequence:client:{id}) was added to Redis, maintained by the Enrichment Service. It stores **unscaled** feature vectors (the Scoring Service scales on-the-fly), ensuring sequences survive model retraining with a new scaler.

13.2 Scoring Service

Consumes from enriched-events, produces to scored-events. Per-event flow: extract 30 features → scale with StandardScaler → pull LSTM sequence from Redis → run ScoreFusion (AE + LSTM + percentile normalization + sigmoid maturity weighting) → if fused_score ≥ 0.95, run SHAP explanations → produce scored event.

Conditional SHAP: KernelExplainer is expensive (~100ms per event). Since ~95% of events score below the REVIEW threshold and receive no explanation, SHAP is gated behind the threshold check. **Model instance sharing:** ScoreFusion loads models once; AEScorer and LSTMScorer receive references via composition, halving memory usage.

13.3 Decision Service (Streaming Wrapper)

The stateless DecisionService engine is wrapped by a streaming adapter that handles Kafka consumption, Redis profile lookup, and message production. The wrapper fetches profile data (archetype, regulatory_client_type, maturity_status, expected_daily_rate) from Redis, builds DecisionInput, delegates to the engine, then attaches event identifiers (client_id, branch_id, timestamp, amount, operation_type) before publishing to decisions. Risk history fields are stubbed pending the supervisor feedback loop.

13.4 Cold Start Resolution

Initial testing produced all scores at 0.999+ regardless of event content. Root cause: empty Redis buffers at startup meant velocity features (tx_count_24h, tx_count_7d, cumulative_amount_24h) were zero for every client, producing feature vectors far outside the training distribution. The fix extends hydrate_redis.py to pre-load client buffers from the recent_events_buffer in each profile snapshot (last 50 events per client), seeding the buffers before streaming services start.

13.5 End-to-End Validation

Metric	Value	Interpretation
Score range	0.002 – 1.000	Full dynamic range, not saturated

Metric	Value	Interpretation
Score mean	0.44	Most normal events well below threshold
Tier distribution	14 INFO / 3 REVIEW / 3 BLOCK	~70% routine, ~30% flagged
Problematic features	0 / 30	All features within training distribution
SHAP explanations	Generated for events $\geq$ 0.95	Conditional path verified
Regulatory flags	Duplicate detection, frequency burst	Rule engine firing correctly

14 Data Flow Summary

Stage	Service	Input	Output	Redis Access	Key Operations
1	Simulator	PostgreSQL	raw-events	—	Replay with client_id partition key
2	Enrichment	raw-events	enriched-events	R: profiles, baselines R/W: buffers, sequences	30 contrast features, rolling buffers
3	Scoring	enriched-events	scored-events	R: LSTM sequences	AE + LSTM fusion, conditional SHAP
4	Decision	scored-events	decisions	R: client profiles	Score tier, 8 rules, explanation assembly

15 Key Technical Learnings

1. Contrast features over profile dumps. Features should measure event-vs-baseline deviation, not repeat the baseline. Profile dominance drowns per-event signal — this single insight took model performance from 0.57 to 0.98.

2. Training-inference symmetry. Models must train on exactly the feature schema they see in production. The oracle shortcut produces the same 30 features as the streaming Enrichment Service.

3. Schema verification before consumer code. Python's dict.get(key, default) makes key mismatches invisible at runtime. Always inspect actual data structure before writing code that assumes key names.

4. Fallback chains for sparse temporal windows. A single 30d window breaks for infrequent operations. The 30d → 90d → archetype baseline chain ensures meaningful comparisons regardless of operation frequency.

5. Store unscaled, scale on-the-fly. LSTM sequence buffers store raw feature vectors. Weekly retraining with a new scaler does not invalidate persisted sequences.

6. Cold start is a production concern. Every pipeline restart clears buffers. Buffer hydration from batch profiles is a required startup step, not an optimization.

7. Conditional computation for throughput. Gating SHAP behind the REVIEW threshold means ~95% of events take the fast path (~5ms vs ~105ms).

16 Artifacts Produced

File	Description
src/generator/generator.py	Rebuilt generator: branch-linked employees, 9 scenarios
src/generator/client.py	Client class with account_opening_date
src/batch/batch_profile.py	Weekly client profiles (122 fields)

File	Description
src/batch/batch_employee_profile.py	Weekly employee profiles + peer comparison
src/batch/batch_archetype_baselines.py	Archetype aggregate baselines
src/batch/batch_enrichment.py	Training data enrichment (74 features)
src/batch/hydrate_redis.py	Profile + buffer hydration into Redis
src/training/data_prep.py	Data preparation with warm-up filter and temporal split
src/training/autoencoder.py	Autoencoder training and evaluation
src/training/lstm.py	LSTM training and evaluation
src/scoring/fusion.py	Score fusion: percentile normalization + sigmoid maturity
src/scoring/ae_scorer.py	AE wrapper with SHAP KernelExplainer
src/scoring/lstm_scorer.py	LSTM wrapper with expected-vs-actual explanation
src/decision/decision.py	3-layer Decision Service with 8 regulatory rules
src/decision/explainer.py	Trigger-aware explanation assembly + NLG templates
src/streaming/simulator.py	Event replay from PostgreSQL to raw-events
src/streaming/enrichment_service.py	raw-events → enriched-events (30 contrast features)
src/streaming/scoring_service.py	enriched-events → scored-events (fusion + SHAP)
src/streaming/decision_service.py	scored-events → decisions (profile + engine)
src/config.py + .env	Centralized configuration
docker-compose.yml	RedPanda + Redis containerization
scripts/test_e2e_pipeline.py	E2E validation script
models/	AE, LSTM, scaler, feature_cols, reference distributions, SHAP background

17 Next Steps

- **Dockerize streaming services:** Enrichment, Scoring, and Decision services added to docker-compose.yml with proper dependency ordering
- **Full Kafka E2E test:** Four terminals, real message flow through all topics
- **Streamlit dashboard:** Consumes decisions topic — alert list with SHAP explanations, per-branch KPIs, simulation mode, supervisor validation feedback loop
- **MLflow + Airflow integration:** Model versioning, experiment tracking, orchestrated weekly retraining including reference score and SHAP background regeneration
- **Batch enrichment rewrite:** Align with 30-feature contrast schema, integrated as Airflow task
- **preferred_day computation:** Add to batch profile, replacing current neutral default